

Kirby 冒险之旅

基于 Qt6 与 C++17 的 2D 横版动作冒险游戏

课程作业报告

小组成员：2 组

开发环境：Qt 6.11 + MinGW 64-bit + C++17；开发工具：Qt Creator

一、项目简介

本项目实现了一款以 Kirby 为主角的横版动作冒险游戏。玩家可以在地图中移动、跳跃、翻滚、吞噬敌人并获得不同能力，完成冒险关卡或挑战 Boss。游戏采用 Qt 的 QGraphicsScene/QGraphicsView 作为场景与视图框架，使用 QTimer 驱动每帧逻辑更新，整体结构围绕 C++ 类、继承、多态、容器和事件处理展开。

修改后的报告保留项目的核心设计与实现说明，删去了未实际完成的机关内容，并把篇幅压缩到约 6 页。报告重点强调本项目与 C++ 程序设计实习课程知识的联系，例如地图方块类 Tile、游戏对象基类 GameObject、敌人继承体系、枚举状态机、动态对象管理和 Qt 事件机制。

主要完成内容

- 完成主菜单、关卡选择、暂停、设置、结束动画等基础流程。
- 完成横版移动、跳跃、翻滚、飞行、受击、生命值和体力条等玩家控制逻辑。
- 完成火、冰、叶、电等复制能力及对应动画、冷却和弹幕效果。
- 完成基于 Tile 方块类的地图搭建、碰撞、水面、冰块、碎石块、木箱与检查点。
- 完成普通敌人与多个 Boss 的行为逻辑、血条、阶段切换和掉落奖励。

二、需求分析与功能设计

游戏目标是提供一个完整可运行的 2D 关卡体验。玩家从主菜单进入冒险模式或 Boss 模式，在地图中通过键盘控制角色，躲避敌人攻击、利用复制能力解决战斗，并在检查点保存进度。由于是课程作业，设计重点不是追求商业游戏规模，而是把 C++ 程序设计、面向对象建模和 Qt 图形界面开发结合起来。

模块	功能说明	涉及课程知识
角色控制	移动、跳跃、翻滚、攻击、吞噬、变身、体力与生命值。	类成员变量、枚举状态、事件处理、封装。
地图系统	地图由不同类型的 Tile 方块组成，统一加入场景并参与碰撞检测。	类设计、构造函数、对象集合、指针管理。
敌人系统	普通敌人继承 Enemy，Boss 继承 BossEnemy，分别实现不同 updateLogic。	继承、多态、虚函数、动态绑定。
战斗系统	玩家和敌人的弹幕使用 Projectile 类，按速度、生命周期和伤害更新。	对象生命周期、容器遍历、碰撞判断。
界面与资源	菜单、按钮、血条、音效和动画帧通过 Qt 图形项与资源文件组织。	Qt 框架、信号槽、资源管理。

玩法流程

- 进入主菜单后选择冒险模式或 Boss 模式，游戏切换到对应关卡。
- MainWindow::loadLevel 根据关卡编号创建地图、玩家、敌人、道具和目标点。
- 游戏循环由 QTimer 触发 gameUpdate，每帧处理输入、运动、碰撞、敌人 AI、弹幕和 UI。
- 玩家到达终点或击败 Boss 后进入奖励、徽章或结束动画流程。

三、总体架构与类设计

项目采用以 MainWindow 为中心的場景管理结构。MainWindow 持有 QGraphicsScene、QGraphicsView、QTimer、玩家对象、地图方块列表、敌人列表、弹幕列表、道具列表和 UI 图形项。这样的设计便于在统一的 gameUpdate 中协调所有对象，同时也让每种对象能够在自己的类中维护状态和行为。

核心类关系

类名	职责	设计要点
GameObject	所有可更新游戏对象的抽象基类。	继承 QGraphicsPixmapItem，声明纯虚函数 updateLogic。
Tile	地图方块类，表示草地、泥土、水、冰块、碎石等地形。	用枚举区分类型，用 changeType 支持地形变化。
Player	玩家角色类。	用 State 和 Element 枚举管理动作与元素能力。
Enemy / BossEnemy	敌人与 Boss 的共同基类。	用虚函数支持不同敌人的移动、受击和吞噬规则。
Projectile	子弹、星星、Boss 弹幕等飞行物。	每帧按 vx、vy 更新位置，并根据生命周期自动清理。
Cake / Star / Crate	道具与交互物。	分别承担回血、攻击、推动解谜等功能。

地图为什么适合用方块类实现

地图中每一个可见地形都抽象为 Tile 对象，类型由 Tile::TileType 枚举表示。关卡数据用字符表示不同方块，加载时按行列换算坐标，创建 Tile 后加入场景，并分别保存到 floors、waters 等容器。这样既符合横版游戏的格子地图特点，也非常适合展示课程中的类、对象、枚举、构造函数、容器和指针操作。

四、关键实现

1. 场景与游戏循环

Qt 的 `QGraphicsScene` 负责保存和绘制所有游戏元素，`QGraphicsView` 负责显示场景区域。项目使用 `QTimer` 定时调用 `gameUpdate`，将游戏运行拆分为输入处理、速度更新、碰撞检测、AI 更新、弹幕更新、UI 更新和对象清理几个步骤。这个结构让游戏逻辑保持连续，也便于在调试时定位每类问题。

2. Tile 地图与碰撞

地图加载时，代码读取关卡字符串中的字符，创建对应的 `Tile` 对象，并设置到 $c * \text{renderSize}$ 、 $r * \text{renderSize}$ 的格子坐标上。固体方块加入 `floors`，水面和水体加入 `waters`。角色、弹幕、木箱等对象移动时会遍历相关方块，先用 `sceneBoundingRect` 做粗略判断，再调用 `collidesWithItem` 做精确碰撞。这里体现了用类封装地形属性、用容器管理对象集合、用几何判断降低无效碰撞计算的思路。

3. 玩家状态与复制能力

`Player` 类用 `State` 枚举表示待机、行走、跳跃、翻滚、攻击、吞噬、消化等动作状态，用 `currentForm` 和 `currentElement` 表示当前能力。火能力可冲刺和爆炸，冰能力可防御并冻结水面，叶能力可释放技能，电能力可高速突进。状态变量和冷却计时器集中在 `Player` 内部，使控制逻辑、动画切换和技能效果之间保持清晰的对应关系。

4. 敌人与 Boss

普通敌人通过 `MinionEnemy` 表示，可携带不同 `CopyAbility`。Boss 类在 `Enemy` 基础上扩展血量、阶段、弹幕和特殊行为。例如 `DukeFishron` 会在不同阶段切换飞行、蓄力、冲刺和弹幕攻击，并能临时改变部分 `Tile` 的类型；`IceGod` 与 `BrainOfCthulhu` 则各自实现自己的弹幕节奏和移动方式。由于都通过 `updateLogic` 参与主循环，因此新增敌人时只需要实现自己的更新策略。

五、课程知识对应

本项目与 C++ 程序设计实习课程的联系比较直接。游戏中的多数功能都不是单独写在一个函数里，而是拆成了有职责边界的类，再通过继承、多态和容器组织起来。下面按课程知识点总结项目中的实际体现。

课程知识点	项目中的体现
类与对象	GameObject、Tile、Player、Enemy、Projectile、Crate 等都对应独立对象，拥有自己的属性和行为。
继承与多态	Player、Enemy、Tile、Projectile 等继承 GameObject；不同敌人和 Boss 重写 updateLogic 与 takeDamage。
枚举类型	TileType、Player::State、Enemy::CopyAbility、GameState 用于表达有限状态，避免大量魔法数字。
构造函数与封装	对象创建时初始化贴图、速度、生命值、状态等，外部通过方法改变对象行为。
指针与动态内存	地图、敌人、弹幕、UI 图形项多为动态创建，清理时需要从场景和容器中同步移除。
STL/Qt 容器	QList、QVector、QSet 用于保存方块、敌人、弹幕、动画帧、按键状态和快照数据。
事件处理	keyPressEvent、keyReleaseEvent、mousePressEvent 和 eventFilter 负责键盘鼠标交互。
模块化编程	每个类拆分为 .h 和 .cpp 文件，接口与实现分离，便于多人协作和调试。

课程知识重点：Tile 方块地图

方块地图是本项目最能体现课程知识的部分。Tile 既是一个图形项，又是一个可参与逻辑更新的游戏对象；它通过枚举保存方块类型，通过构造函数接收贴图，通过 changeType 改变地形状态。MainWindow 只需要根据字符地图批量创建 Tile，并把它们放入不同容器，就能让后续的碰撞、水面、冰块和 Boss 地形变化都复用同一套对象模型。

六、测试、问题与总结

测试内容

- 测试主菜单、关卡选择、暂停、设置和返回主菜单等流程是否能正常切换。
- 测试玩家移动、跳跃、翻滚、吞噬、变身、技能冷却、受击无敌和死亡重开。
- 测试 Tile 方块碰撞、水面交互、冰块转换、木箱推动和检查点复活。
- 测试普通敌人巡逻、攻击、被吞噬、掉落能力，以及 Boss 阶段切换和弹幕清理。
- 测试资源加载、音效播放、血条显示、结束动画和窗口固定尺寸下的画面显示。

遇到的问题与解决

开发中最常见的问题是对象数量多以后，碰撞与清理容易互相影响。例如弹幕、敌人、方块和道具都在场景中，如果只删除场景项而不更新容器，就可能出现悬空指针；如果只更新容器而不从场景移除，画面又会留下无效对象。解决方法是在删除对象时同时维护 scene 和 QList，并在遍历时采用倒序或先记录再删除的方式降低出错概率。

另一个问题是角色状态较多，动画、输入和技能效果容易交叉。项目中通过 State 枚举、冷却计时器和布尔状态变量把逻辑拆开，使每个能力只处理自己的触发条件和持续时间。虽然代码仍有继续整理空间，但已经能支撑完整的游戏流程。

项目总结

通过本次实习项目，我们把 C++ 类设计、继承多态、容器、指针、事件处理和 Qt 图形界面开发结合到一个完整游戏中。尤其是 Tile 方块地图的实现，让地图编辑、碰撞、地形变化和关卡加载都建立在统一的对象模型上。项目最终完成了从菜单到关卡、从普通敌人到 Boss、从玩家操作到音画反馈的基本闭环，达到了课程作业对综合实践能力的要求。